CMG–PES Residual Control of UAV Quaternion Tracking

Full derivations, cited fundamentals, and equation-to-code map
companion to the ACC 2027 manuscript (v9, 2026-09-12)

Gandhi Contreras (personal working document)

September 12, 2026

How to read this document

- **Paper references.** "P-(11)" is equation (11) of the paper; Proposition 1, Theorem 1, Table II likewise.
- **Code references.** Every listing is read from the file at the quoted line range. Paths are relative to the repository root `C:/Research/papers/ACC2027_Gate_Residual_Control`. File hashes are in Appendix A.
- **Status tags.** *Standard* results carry a citation; *derived here* steps are written out in full; *verified* numbers come from `experiments/theory_certificate_2026-09-12/compute_local_certificate.py`, whose output also generates the paper's macros (`certificate_macros.tex`).
- **Two implementations.** The direct NumPy controller (`src/gra_control/gate.py`) and the Nengo realization (`experiments/.../nengo_cmg.py`) implement the same equations; the paper's tables use the Nengo runner `run_mujoco_nef.py`.

Contents

1 Quaternion algebra, kinematics, and attitude error **3**
1.1 Hamilton product, conjugate, rotation . . . 3
1.2 Active and passive kinematics P-(2)–P-(3) . . . 3
1.3 Error quaternion and its kinematics P-(6)–P-(8) . . . 4
1.4 References (Euler 3-2-1 to quaternion) . . . 4

2 Rigid body, inertia frame, and the PD baseline **5**
2.1 Body tensor from principal moments P-(9) . . . 5
2.2 Why $k_q = 2j\omega_n^2$ and $k_\omega = 2\zeta j\omega_n$ P-(11) . . . 5
2.3 Torque box . . . 6

3 Allocation and the height loop **6**
3.1 Allocation matrix P-(12) . . . 6
3.2 Clipped inverse allocation P-(13) . . . 7
3.3 Collective thrust P-(14) . . . 7

4 The coupled membrane–gate unit **7**
4.1 Encoding P-(15) . . . 7
4.2 Continuous dynamics P-(18) and the frozen-coefficient membrane step P-(21) . . . 8
4.3 Gate step P-(20): implemented forward Euler vs. exponential Euler . . . 8
4.4 Proposition 1 (bounded internal states), full proof . . . 9
4.5 Rest point, resting activity, and gate latching (Sec. IV-D) . . . 9

5 Readout, PES learning, and the bounded residual **9**
5.1 Readout P-(22) . . . 9
5.2 PES as a local gradient step P-(25) . . . 10
5.3 Nengo learning-rate conversion . . . 10
5.4 Proposition 2 (bounded authority), full proof . . . 11
5.5 Proposition 3 (restricted contraction), full proof, and the clip lemma . . . 11

6 Latched regime: lagged proportional action and σ-modification **11**
6.1 Constant-basis recursion P-(36) . . . 11
6.2 Pole, time constant, DC gain, frequency response . . . 11
6.3 σ-modification fixed point . . . 12
6.4 Why silent bases do not learn (Sec. IV-D, neuron families) . . . 12

7 Allocation-aware local stability (Sec. III-D) **13**
7.1 Secant lemma and L_A P-(29) . . . 13
7.2 Local model P-(30)–P-(31) . . . 13
7.3 Quadratic-growth constant L_r P-(32) . . . 13
7.4 A is Hurwitz (energy + LaSalle) . . . 14
7.5 Theorem 1, full proof . . . 14
7.6 Optimal radius, maximal admissible disturbance, and $\beta_{\max}$ P-(35) . . . 14
7.7 Remark (sign selection on certified trajectories) . . . 15

8 Protocol, metrics, noise, and disturbance **16**
8.1 Metric . . . 16
8.2 Campaign settings . . . 16
8.3 Measurement noise (Sec. IV-C) . . . 16
8.4 Disturbance (Sec. IV-F) . . . 16

9 What changed relative to the highlighted v7, mathematically **17**

A Source files and hashes **18**

1 Quaternion algebra, kinematics, and attitude error

1.1 Hamilton product, conjugate, rotation

For $p = [p_0, \boldsymbol{p}^\mathsf{T}]^\mathsf{T}$, $q = [q_0, \boldsymbol{q}^\mathsf{T}]^\mathsf{T}$ (scalar first) the Hamilton product is

$$p \otimes q = \begin{bmatrix} p_0 q_0 - \boldsymbol{p}^\mathsf{T}\boldsymbol{q} \\ p_0\boldsymbol{q} + q_0\boldsymbol{p} + \boldsymbol{p} \times \boldsymbol{q} \end{bmatrix}, \tag{1}$$

associative but not commutative; $q^* = [q_0, -\boldsymbol{q}^\mathsf{T}]^\mathsf{T}$ and, for unit q, $q^{-1} = q^*$. A unit quaternion rotates vectors by $R(q)\boldsymbol{v} = \mathrm{vec}(q \otimes [0, \boldsymbol{v}^\mathsf{T}]^\mathsf{T} \otimes q^*)$, so $R(q)^\mathsf{T}\boldsymbol{v} = R(q^*)\boldsymbol{v} = \mathrm{vec}(q^* \otimes [0, \boldsymbol{v}^\mathsf{T}]^\mathsf{T} \otimes q)$ [1, 2].

Status. Standard.

`src/gra_control/math3d.py`: 12–25 conjugate and Hamilton product (1)

```python
def quat_conj(q: np.ndarray) -> np.ndarray:
    q = np.asarray(q, dtype=float)
    return np.array([q[0], -q[1], -q[2], -q[3]], dtype=float)


def quat_mul(p: np.ndarray, q: np.ndarray) -> np.ndarray:
    p0, p1, p2, p3 = p
    q0, q1, q2, q3 = q
    return np.array([
        p0*q0 - p1*q1 - p2*q2 - p3*q3,
        p0*q1 + p1*q0 + p2*q3 - p3*q2,
        p0*q2 - p1*q3 + p2*q0 + p3*q1,
        p0*q3 + p1*q2 - p2*q1 + p3*q0,
    ], dtype=float)
```

1.2 Active and passive kinematics P-(2)–P-(3)

With Q the body-to-inertial quaternion and $\boldsymbol{\omega}$ the body rate, $\dot{Q} = \frac{1}{2} Q \otimes \tilde{\boldsymbol{\omega}}$ with $\tilde{\boldsymbol{\omega}} = [0, \boldsymbol{\omega}^\mathsf{T}]^\mathsf{T}$ [2]. For the passive $q = Q^{-1} = Q^*$: differentiating $Q \otimes Q^* = 1$ gives $\dot{Q} \otimes Q^* + Q \otimes (\dot{Q}^*) = 0$, hence

$$\dot{q} = (\dot{Q}^*) = -Q^* \otimes \dot{Q} \otimes Q^* = -\tfrac{1}{2}\, Q^* \otimes Q \otimes \tilde{\boldsymbol{\omega}} \otimes Q^* = -\tfrac{1}{2}\, \tilde{\boldsymbol{\omega}} \otimes q,$$

which is P-(3). *Derived here.*

Code convention. MuJoCo stores the free-joint quaternion Q (body→world) in `qpos`; the runner forms $q = Q^*$ and reads $\boldsymbol{\omega}$ in body axes from `qvel`:

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py`: 216–230 measurement, optional noise, reference, error, PD, PES target

```python
        while data.time < args.duration:
            t = float(data.time)
            q_meas = quat_conj(np.asarray(data.qpos[qa + 3 : qa + 7], dtype=float))
            omega = np.asarray(data.qvel[va + 3 : va + 6], dtype=float).copy()
            if args.attitude_noise_std > 0:
                dq = euler_to_quat(*rng.normal(0.0, args.attitude_noise_std, size=3))
                q_meas = quat_mul(dq, q_meas)
            if args.rate_noise_std > 0:
                omega += rng.normal(0.0, args.rate_noise_std, size=3)

            ref_euler = reference_euler(t, args.mission, args.duration)
            qref = reference_quat(t, args.mission, args.duration)
            qe, eq = quat_error(qref, q_meas)
            tau0 = np.clip(-kq * eq - kw * omega, -limits, limits)
            target = np.clip(-args.k_error * eq - args.k_rate * omega, -1.0, 1.0)
```

1.3 Error quaternion and its kinematics P-(6)–P-(8)

Let $q_e = q_d \otimes q^{-1}$ with $\dot{q}_d = -\frac{1}{2}\tilde{\boldsymbol{\omega}}_d \otimes q_d$ (same passive convention). Using $\dot{(q^*)} = \frac{1}{2} q^* \otimes \tilde{\boldsymbol{\omega}}$ (same argument as above),

$$\dot{q}_e = \dot{q}_d \otimes q^* + q_d \otimes \dot{(q^*)} = -\tfrac{1}{2}\tilde{\boldsymbol{\omega}}_d \otimes q_e + \tfrac{1}{2} q_e \otimes \tilde{\boldsymbol{\omega}}.$$

Insert $q_e \otimes q_e^* = 1$: $\tilde{\boldsymbol{\omega}}_d \otimes q_e = q_e \otimes (q_e^* \otimes \tilde{\boldsymbol{\omega}}_d \otimes q_e) = q_e \otimes [0, (R(q_e)^{\mathsf{T}} \boldsymbol{\omega}_d)^{\mathsf{T}}]^{\mathsf{T}}$. Therefore

$$\dot{q}_e = \tfrac{1}{2} q_e \otimes [0, \boldsymbol{\omega}_e^{\mathsf{T}}]^{\mathsf{T}}, \qquad \boldsymbol{\omega}_e = \boldsymbol{\omega} - R(q_e)^{\mathsf{T}} \boldsymbol{\omega}_d. \tag{2}$$

Taking scalar and vector parts of (1):

$$\dot{q}_{e,0} = -\tfrac{1}{2} \boldsymbol{q}_{e,v}^{\mathsf{T}} \boldsymbol{\omega}_e, \qquad \dot{\boldsymbol{q}}_{e,v} = \tfrac{1}{2}\big(q_{e,0} \boldsymbol{\omega}_e + \boldsymbol{q}_{e,v} \times \boldsymbol{\omega}_e\big). \tag{3}$$

The first is P-(8); the second is used in Sec. 7. *Derived here.*

Shortest-path error. Write $q_e = [\cos\frac{\theta}{2},\ \sin\frac{\theta}{2}\,\boldsymbol{n}^{\mathsf{T}}]^{\mathsf{T}}$. Then $\|\boldsymbol{q}_{e,v}\| = |\sin\frac{\theta}{2}|$ and $|q_{e,0}| = |\cos\frac{\theta}{2}|$, so $\theta_q = 2\,\mathrm{atan2}(\|\boldsymbol{q}_{e,v}\|, |q_{e,0}|) \in [0, \pi]$ is the smallest rotation angle and is invariant under $q_e \mapsto -q_e$. Sign selection $\boldsymbol{e}_q = \mathrm{sgn}_+(q_{e,0}) \boldsymbol{q}_{e,v}$ picks the hemisphere $q_{e,0} \geq 0$; it is discontinuous at $q_{e,0} = 0$, and no continuous feedback can globally stabilize attitude [3, 4]; robust global tracking needs hybrid [5] or geometric-algebra [6, 7] designs.

src/gra_control/math3d.py : 40–51 error quaternion, sgn_+ (`if qe[0] < 0`), angle θ_q

```python
def quat_error(q_ref: np.ndarray, q_meas: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Return shortest-path q_ref * conj(q_meas) and its vector part."""
    qe = quat_normalize(quat_mul(quat_normalize(q_ref), quat_conj(quat_normalize(q_meas))))
    eq = qe[1:].copy()
    if qe[0] < 0:
        eq *= -1.0
    return qe, eq


def quat_angle(qe: np.ndarray) -> float:
    qe = quat_normalize(qe)
    return float(2.0*np.arctan2(np.linalg.norm(qe[1:]), max(abs(qe[0]), 1e-12)))
```

1.4 References (Euler 3-2-1 to quaternion)

The reference Euler triple (ϕ, ϑ, ψ) is mapped by the standard 3-2-1 half-angle formula [1]; missions are the closed forms of Sec. IV-A.

src/gra_control/math3d.py : 28–37 3-2-1 Euler to quaternion

```python
def euler_to_quat(phi: float, theta: float, psi: float) -> np.ndarray:
    cr, sr = np.cos(phi/2), np.sin(phi/2)
    cp, sp = np.cos(theta/2), np.sin(theta/2)
    cy, sy = np.cos(psi/2), np.sin(psi/2)
    return quat_normalize(np.array([
        cr*cp*cy + sr*sp*sy,
        sr*cp*cy - cr*sp*sy,
        cr*sp*cy + sr*cp*sy,
        cr*cp*sy - sr*sp*cy,
    ]))
```

src/gra_control/math3d.py : 54–82 mission references (Roll, Full3, Hard45, Hardstep, Yaw360)

```python
def reference_euler(t: float, mission: str, duration: float) -> np.ndarray:
    if mission == "hover":
        return np.zeros(3)
    if mission == "roll":
```

```
        return np.array([0.25*np.sin(0.7*t), 0.0, 0.0])
    if mission == "full3":
        return 0.5*np.sin(t + np.array([0.0, np.pi/2, np.pi]))
    if mission == "hard45":
        return np.array([
            0.55*np.sin(0.75*t) + 0.20*np.sin(2.10*t + 0.40),
            0.45*np.sin(0.95*t + np.pi/3) + 0.18*np.sin(1.75*t),
            0.65*np.sin(0.55*t + np.pi) + 0.25*np.sin(1.35*t + 0.20),
        ])
    if mission == "hardstep":
        if t < 1.0:
            return np.zeros(3)
        phase = int((t-1.0)//2.0) % 4
        return np.array([
            [0.75, -0.45, 0.55], [-0.65, 0.55, -0.70],
            [0.50, 0.50, 0.90], [-0.80, -0.35, 0.20],
        ][phase])
    if mission == "yaw360":
        s = np.clip(t/max(duration, 1e-9), 0.0, 1.0)
        return np.array([0.0, 0.0, 2*np.pi*(3*s*s-2*s*s*s)])
    raise ValueError(f"Unknown mission: {mission}")


def reference_quat(t: float, mission: str, duration: float) -> np.ndarray:
    return euler_to_quat(*reference_euler(t, mission, duration))
```

2 Rigid body, inertia frame, and the PD baseline

2.1 Body tensor from principal moments P-(9)

MuJoCo stores principal moments `body_inertia` $= \boldsymbol{j}_{\mathrm{p}}$ and the principal-frame orientation `body_iquat`. The body-frame tensor is $\boldsymbol{J} = R_{BI}\,\mathrm{diag}(\boldsymbol{j}_{\mathrm{p}})R_{BI}^{\mathsf{T}}$ with $R_{BI} = R($`body_iquat`$)$ (similarity transform of a symmetric tensor, [2]).

Status. Verified: the certificate script reproduces the paper's X2 tensor $10^{-2}[3.66517, 0, -.21;\ 0, 2.54117, 0;\ -.21, 0, 6.$ from the model file.

`experiments/theory_certificate_2026-09-12/compute_local_certificate.py`:92–101 $\boldsymbol{J}$ from `body_iquat` and `body_inertia`

```
def analyse(platform, scale, rng):
    spec = SPECS[platform]
    model = mujoco.MjModel.from_xml_path(str(spec.model))
    _, body, _, _ = _free_joint(model)
    j = model.body_inertia[body].copy()                        # j_tune (principal moments)
    J = quat_to_rot(model.body_iquat[body]) @ np.diag(j) @ quat_to_rot(model.body_iquat[body]).T
    mass = float(np.sum(model.body_mass))
    g = abs(float(model.opt.gravity[2]))
    ell = torque_limits(platform, j, model, "model")
    kq, kw = 2 * j * WN**2, 2 * ZETA * j * WN
```

2.2 Why $k_q = 2j\omega_n^2$ and $k_\omega = 2\zeta j\omega_n$ P-(11)

For one principal axis with small error, $e_q \approx \frac{\theta}{2}$ and (from (3) with $q_{e,0} \approx 1$) $\dot{e}_q \approx \frac{1}{2}\omega$. With $j\dot{\omega} = -k_q e_q - k_\omega\omega$ and $\omega = \dot{\theta}$:

$$j\ddot{\theta} + k_\omega\dot{\theta} + \tfrac{k_q}{2}\theta = 0 \quad\Longrightarrow\quad \omega_n^2 = \frac{k_q}{2j}, \qquad 2\zeta\omega_n = \frac{k_\omega}{j},$$

so $k_q = 2j\omega_n^2$ and $k_\omega = 2\zeta j\omega_n$. The factor 2 compensates the half angle. With $\boldsymbol{J}$ not diagonal and $\boldsymbol{j}_{\mathrm{tune}} \neq \mathrm{diag}(\boldsymbol{J})$ (X2), ω_n is a tuning label, not an exact closed-loop frequency (as the paper states). *Derived here.*

experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py : 159–165 inertia, limits, gains

```
    inertia = model.body_inertia[body_id].copy()
    mass = float(np.sum(model.body_mass))
    gravity = abs(float(model.opt.gravity[2]))
    limits = torque_limits(args.platform, inertia, model, args.torque_policy)
    wn = np.array([4.0, 4.0, 3.5])
    kq = 2.0 * inertia * wn * wn
    kw = 2.0 * 0.95 * wn * inertia
```

2.3 Torque box

$\ell_i = \max(j_{\text{tune},i}\nu_i, 10^{-6})$ with $\boldsymbol{\nu} = (12, 12, 10)\,\text{rad/s}^2$: the torque that gives each axis a ν_i angular acceleration. The campaign uses `-torque-policy model`, which for the rotor-interface platforms returns exactly this inertia limit.

src/gra_control/platforms/drone.py : 76–98 torque_limits

```
def torque_limits(
    platform_name: str,
    inertia: np.ndarray,
    model: mujoco.MjModel,
    policy: str,
) -> np.ndarray:
    inertia_limit = np.maximum(inertia*np.array([12.0, 12.0, 10.0]), 1e-6)
    if policy == "inertia":
        return inertia_limit
    if policy == "legacy-learned":
        return np.maximum(inertia_limit, np.array([0.03, 0.03, 0.02]))
    if policy == "model":
        if platform_name == "cf2":
            names = ["x_moment", "y_moment", "z_moment"]
            aids = np.array([_name_id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, n) for n in names])
            gear = np.abs(np.array([
                model.actuator_gear[aids[0], 3], model.actuator_gear[aids[1], 4],
                model.actuator_gear[aids[2], 5],
            ]))
            ctrl = np.max(np.abs(model.actuator_ctrlrange[aids]), axis=1)
            return np.minimum(inertia_limit, gear*ctrl)
        return inertia_limit
    raise ValueError(f"Unknown torque policy: {policy}")
```

3 Allocation and the height loop

3.1 Allocation matrix P-(12)

A rotor at $\boldsymbol{p}_i = (p_{i,x}, p_{i,y}, p_{i,z})$ with thrust $[0, 0, f_i]^\mathsf{T}$ produces torque $\boldsymbol{p}_i \times [0, 0, f_i]^\mathsf{T} = [p_{i,y} f_i,\ -p_{i,x} f_i,\ 0]^\mathsf{T}$; the drag reaction adds $\kappa_i f_i$ about z [8]. Summing over rotors gives $[T, \boldsymbol{\tau}^\mathsf{T}]^\mathsf{T} = B\boldsymbol{f}$ with rows $(1,\ p_{i,y},\ -p_{i,x},\ \kappa_i)$. For four rotors B is square and invertible, so $B^{-1} = B^\dagger$.

src/gra_control/platforms/drone.py : 67–73 B from site positions and gear, `np.linalg.inv(B)`

```
def _mixer(model: mujoco.MjModel) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    names = [f"thrust{i}" for i in range(1, 5)]
    aids = np.array([_name_id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, n) for n in names])
    sids = np.array([_name_id(model, mujoco.mjtObj.mjOBJ_SITE, n) for n in names])
    pos = model.site_pos[sids]
    B = np.vstack([np.ones(4), pos[:, 1], -pos[:, 0], model.actuator_gear[aids, 5]])
    return aids, B, np.linalg.inv(B), model.actuator_ctrlrange[aids].copy()
```

3.2 Clipped inverse allocation P-(13)

experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py : 309–314 $\boldsymbol{f}_{\text{req}} = B^{-1}[T, \boldsymbol{u}^{\mathsf{T}}]^{\mathsf{T}}$, clip to $[0, \boldsymbol{f}_{\max}]$

```
            if spec.interface == "rotor4":
                actuator_command = B_inv @ np.r_[collective, tau]
                actuator_command = np.clip(
                    actuator_command, ctrl_range[:, 0], ctrl_range[:, 1]
                )
                data.ctrl[act_ids] = actuator_command
```

The CF2 authority scale multiplies only the upper force limit:

experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py : 183–188 actuator scale

```
    if spec.interface == "rotor4":
        act_ids, _, B_inv, ctrl_range = _mixer(model)
        if args.actuator_scale <= 0:
            raise ValueError("actuator_scale must be positive")
        ctrl_range[:, 1] *= args.actuator_scale
        model.actuator_ctrlrange[act_ids, 1] = ctrl_range[:, 1]
```

3.3 Collective thrust P-(14)

Vertical force balance: $TR(Q)_{33} - Mg = M\ddot{z}$. Imposing $\ddot{z} = 6(z_d - z) - 4\dot{z}$ gives $T = M[g + 6(z_d - z) - 4\dot{z}]/R_{33}$, a PD altitude loop with $\omega_{n,z} = \sqrt{6} = 2.449\,\text{rad/s}$ and $\zeta_z = 4/(2\sqrt{6}) = 0.816$. The guard $R_{33} \geq 0.35$ caps the tilt compensation at $\arccos(0.35) = 69.5°$ (factor ≤ 2.86), and $T \leq 2.5Mg$. *Derived here.*

experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py : 289–295 collective thrust

```
            R = np.asarray(data.xmat[body_id], dtype=float).reshape(3, 3)
            z, vz = float(data.qpos[qa + 2]), float(data.qvel[va + 2])
            tilt = max(float(R[2, 2]), 0.35)
            collective = mass * (gravity + 6.0 * (1.0 - z) - 4.0 * vz) / tilt
            collective = float(
                np.clip(collective, 0.0, 2.5 * mass * max(gravity, 1.0))
            )
```

4 The coupled membrane–gate unit

4.1 Encoding P-(15)

$\bar{\boldsymbol{u}} = \boldsymbol{u}_0 \oslash \boldsymbol{\ell} \in [-1, 1]^3$ (because $\boldsymbol{u}_0$ is already saturated), and $\boldsymbol{I}^{\pm} = I_b\mathbf{1} + I_g \max(\pm\bar{\boldsymbol{u}}, 0) \in [I_b, I_b + I_g]^3$. A pair never has both currents above I_b: one sign carries the request, the other rests.

experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py : 231–239 normalized request and paired currents (logging copy)

```
            normalized = np.divide(
                tau0, limits, out=np.zeros_like(tau0), where=limits > 1e-12
            )
            current = np.concatenate(
                [
                    0.75 + 1.25 * np.maximum(normalized, 0.0),
                    0.75 + 1.25 * np.maximum(-normalized, 0.0),
                ]
            )
```

`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py` : 318–330 the same encoding inside the Nengo augmentor

```
        u0 = np.clip(np.asarray(baseline, dtype=float), -self.limits, self.limits)
        normalized = np.divide(
            u0,
            self.limits,
            out=np.zeros_like(u0),
            where=self.limits > 1e-12,
        )
        self.external.current[:] = np.concatenate(
            [
                self.current_bias + self.current_gain * np.maximum(normalized, 0.0),
                self.current_bias + self.current_gain * np.maximum(-normalized, 0.0),
            ]
        )
```

4.2 Continuous dynamics P-(18) and the frozen-coefficient membrane step P-(21)

The model is conductance-based in the Hodgkin–Huxley sense [9]: a gate r_j relaxes to a logistic target set by the membrane, and a gate-dependent shunt conductance pulls the membrane toward a reversal level. Rewrite the membrane equation as an affine ODE:

$$\dot{v} = -\underbrace{\left(\tfrac{1}{\tau} + c\right)}_{\lambda} v + \underbrace{\tfrac{v_{\text{rest}}+I}{\tau} + c\, v_{\text{reset}}}_{d}.$$

With λ, d frozen over $[t_k, t_k + h]$, the integrating factor $e^{\lambda t}$ gives $\frac{d}{dt}(e^{\lambda t} v) = e^{\lambda t} d$, hence

$$v(t_k + h) = e^{-\lambda h} v_k + (1 - e^{-\lambda h})\, \frac{d}{\lambda},$$

exactly P-(21) (an exponential integrator [10]). Because $0 < e^{-\lambda h} < 1$, this is a convex combination of v_k and the frozen equilibrium d/λ. *Derived here.*

4.3 Gate step P-(20): implemented forward Euler vs. exponential Euler

Implemented. $r^+ = \text{clip}_{[0,1]}\left[(1 - \frac{h}{\tau_r})r + \frac{h}{\tau_r} b\right]$. For $0 < h \le \tau_r$ the weights are nonnegative and sum to one, so with $r, b \in [0,1]$ the argument is already in $[0,1]$ and the clip is inactive. For $h > \tau_r$ the weight $1 - h/\tau_r$ is negative and the clip is what bounds r. The paper uses $h/\tau_r = 0.1$.

Exponential Euler (Dr. Sandoval's proposal). Freezing b, $\dot{r} = (b - r)/\tau_r$ is solved exactly by $r^+ = e^{-h/\tau_r} r + (1 - e^{-h/\tau_r}) b$, convex for every $h > 0$, so no clip is needed [10]. At $h/\tau_r = 0.1$ the weights are 0.9048/0.0952 vs. 0.9/0.1; the two schemes agree to $O((h/\tau_r)^2)$ per step. All reported results use the implemented step.

`src/gra_control/gate.py` : 41–55 direct CMG bank: gate (Euler + clip), then membrane (exact frozen step)

```
    def step(self, current: np.ndarray, dt: float) -> np.ndarray:
        p = self.cfg
        current = np.asarray(current, dtype=float)
        x = np.clip(p.alpha*(self.v-(p.v_threshold-p.gamma*self.r)), -60.0, 60.0)
        target = 1.0/(1.0+np.exp(-x))
        self.r += dt*(target-self.r)/p.tau_r
        self.r[:] = np.clip(self.r, 0.0, 1.0)

        gain = p.base_gain + (p.gain_scaling*np.abs(current) if self.adaptive else 0.0)
        conductance = gain*self.r*self.r
        lam = 1.0/p.tau + conductance
        drive = p.v_rest/p.tau + current/p.tau + conductance*p.v_reset
        decay = np.exp(-lam*dt)
        self.v[:] = self.v*decay + (drive/np.maximum(lam, 1e-12))*(1.0-decay)
        return self.r.copy()
```

`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py`: 71–89 Nengo `NeuronType.step`: identical update, activity clip(Gr)

```
    def step(self, dt, J, output, voltage, gate):
        x = np.clip(
            self.alpha * (voltage - (self.v_threshold - self.gamma * gate)),
            -60.0,
            60.0,
        )
        gate_target = 1.0 / (1.0 + np.exp(-x))
        gate += dt * (gate_target - gate) / self.tau_r
        np.clip(gate, 0.0, 1.0, out=gate)

        gain = self.base_gain
        if self.adaptive:
            gain = gain + self.gain_scaling * np.abs(J)
        conductance = gain * gate * gate
        lam = 1.0 / self.tau + conductance
        drive = self.v_rest / self.tau + J / self.tau + conductance * self.v_reset
        decay = np.exp(-lam * dt)
        voltage[:] = voltage * decay + (drive / np.maximum(lam, 1e-12)) * (1.0 - decay)
        output[:] = np.clip(self.gate_gain * gate, 0.0, 1.0)
```

Order matters. The code updates r first and uses the *updated* r^+ in c; the paper states $c_{j,k} = (g_0 + \chi g_1 |I|)(r^+)^2$ accordingly.

4.4 Proposition 1 (bounded internal states), full proof

Gate: as above. *Membrane:* $I_j \in [I_b, I_b + I_g]$ and $c_j \geq 0$, so $d/\lambda = \frac{1/\tau}{1/\tau + c}(v_{\text{rest}} + I) + \frac{c}{1/\tau + c} v_{\text{reset}}$ is a convex combination of $v_{\text{rest}} + I \in [v_{\text{rest}} + I_b, v_{\text{rest}} + I_b + I_g]$ and v_{reset}. The update is a convex combination of v_k and d/λ. By induction v_k stays in the convex hull of $\{v_0, v_{\text{reset}}\} \cup [v_{\text{rest}} + I_b, v_{\text{rest}} + I_b + I_g]$. □

4.5 Rest point, resting activity, and gate latching (Sec. IV-D)

Rest. With $\bar{u} = 0$, $I = I_b = 0.75$, and $r \approx 0$ (so $c \approx 0$): $v^\star = v_{\text{rest}} + I_b = 0.75$. The gate target is $b = \sigma(\alpha_h(v^\star - v_{\text{th}} + \gamma r)) \approx \sigma(50 \cdot (-0.25)) = \sigma(-12.5) = 3.7267 \times 10^{-6}$, and $r^\star \approx b$ (the γr correction is $O(10^{-4})$ in the argument). The resting activity is $a^\star = \min(1, Gr^\star) = 10^5 \cdot 3.7267 \times 10^{-6} = 0.3727$.

Latch. Suppose $r_k \geq \frac{1}{2}$ and $v_k \geq 0$ (Proposition 1 gives $v \geq 0$ here since $v_{\text{reset}} = v_{\text{rest}} = 0$). Then $\alpha_h(v - v_{\text{th}} + \gamma r) \geq \alpha_h(0 - 1 + 2 \cdot \frac{1}{2}) = 0$, so $b_k \geq \frac{1}{2}$, and the convex Euler step keeps $r_{k+1} \geq \frac{1}{2}$. The set $\{r \geq \frac{1}{2}\}$ is forward invariant: an opened gate cannot close.

Shunted membrane. With $r \approx 1$ and unexcited $I = 0.75$: $c = g_0 + g_1 I = 120 + 22.5 = 142.5$, $\lambda = 20 + 142.5$, $d = I/\tau = 15$, so $v^\star = 15/162.5 = 0.0923$ ("v_j near 0.09"). *Derived here; numbers match the archived traces.*

5 Readout, PES learning, and the bounded residual

5.1 Readout P-(22)

$\boldsymbol{a} = \text{clip}_{[0,1]}(G\boldsymbol{r}^+)$ saturates once $r \geq 1/G = 10^{-5}$. $W_0 = [I_3, -I_3]$ gives $\boldsymbol{s} = \text{clip}(\boldsymbol{a}^+ - \boldsymbol{a}^-)$: pair subtraction recovers a signed quantity from nonnegative features.

`src/gra_control/gate.py`: 104–134 direct augmentor: encoding, readout, PES, residual

```
    def step(
        self,
        baseline: np.ndarray,
        dt: float,
        learning_target: np.ndarray | None = None,
    ) -> AugmentorOutput:
```

```
        u0 = np.clip(np.asarray(baseline, dtype=float), -self.limits, self.limits)
        normalized = np.divide(u0, self.limits, out=np.zeros_like(u0), where=self.limits > 1e-12)
        current = np.concatenate([
            self.cfg.current_bias + self.cfg.current_gain*np.maximum(normalized, 0.0),
            self.cfg.current_bias + self.cfg.current_gain*np.maximum(-normalized, 0.0),
        ])
        gate = self.units.step(current, dt)
        activity = np.clip(self.cfg.gate_gain*gate, 0.0, 1.0)
        signed = np.clip(self.W@activity, -1.0, 1.0)

        if self.mode == "online_pes" and learning_target is not None:
            target = np.clip(np.asarray(learning_target, dtype=float), -1.0, 1.0)
            self.W += self.learning_rate*np.outer(target-signed, activity) - self.leak*self.W
            self.W[:] = np.clip(self.W, -6.0, 6.0)

        neural = signed*self.limits
        if self.mode == "blend":
            command = (1.0-self.beta)*u0 + self.beta*neural
            residual = command-u0
        else:
            residual = self.beta*neural
            command = u0+residual
        command = np.clip(command, -self.limits, self.limits)
        residual = command-u0
        return AugmentorOutput(command, u0, residual, activity, gate)
```

5.2 PES as a local gradient step P-(25)

PES [11, 12] descends $E = \frac{1}{2}\|\boldsymbol{y}^* - W\boldsymbol{a}\|^2$ in W: $-\partial E/\partial W = (\boldsymbol{y}^* - W\boldsymbol{a})\boldsymbol{a}^\mathsf{T} = \boldsymbol{\varepsilon}\boldsymbol{a}^\mathsf{T}$, so $W \leftarrow W + \eta\boldsymbol{\varepsilon}\boldsymbol{a}^\mathsf{T}$. The code uses the clipped readout inside $\boldsymbol{\varepsilon}$ (a projected-output variant). The leak $-\lambda_W W$ is the σ-modification of robust adaptive control [13, 14], and $\Pi_\mathcal{W}$ is elementwise clipping to $[-6, 6]$. The residual uses W_k *before* the update (`signed` is computed first).

5.3 Nengo learning-rate conversion

Nengo 4.1.0's `SimPES` applies $\Delta W = \alpha\, \boldsymbol{e}\, \boldsymbol{a}^\mathsf{T}$ with $\alpha = -\kappa\, dt/n$ (source: `nengo/builder/learning_rules.py`, `SimPES.make_step`; n = number of presynaptic neurons) and error $\boldsymbol{e} = \boldsymbol{s} - \boldsymbol{y}^* = -\boldsymbol{\varepsilon}$. Hence $\Delta W = (\kappa\, dt/n)\, \boldsymbol{\varepsilon}\boldsymbol{a}^\mathsf{T}$; matching $\eta\boldsymbol{\varepsilon}\boldsymbol{a}^\mathsf{T}$ requires $\kappa = \eta n/dt = 8 \times 10^{-4} \cdot 6/5 \times 10^{-4} = 9.6$. Leak (on pre-step weights) and projection are applied after the simulator step.

`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py` : 147–150 $\quad \eta_N = \eta\, n/dt$

```
        # Nengo's SimPES applies -eta_N * dt/n * e a^T.  This conversion makes
        # that increment equal to the frozen controller's
        # +eta_direct * (target-decoded) a^T.
        self.nengo_learning_rate = self.learning_rate * self.n_units / self.dt
```

`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py` : 335–359 step: leak on pre-step weights, clip, residual

```
        weights_before = self._weights.copy()
        self.sim.step()

        self._last_decoded[:] = np.asarray(self._decoded, dtype=float)
        self._last_error[:] = np.asarray(self._error, dtype=float)

        # Preserve the frozen controller's decoder regularization.  PES itself
        # is Nengo's built-in learning operator; only leak and safety clipping
        # are imposed here after its update.
        # Direct semantics are W_new = W_old + PES - leak*W_old.  Subtracting
        # from the pre-step value avoids scaling the just-computed PES update.
        self._weights[:] -= self.leak * weights_before
        np.clip(self._weights, -6.0, 6.0, out=self._weights)

```

```
        signed = np.clip(self._last_decoded, -1.0, 1.0)
        residual_requested = self.beta * signed * self.limits
        command = np.clip(u0 + residual_requested, -self.limits, self.limits)
        residual = command - u0
        return NengoAugmentorOutput(
            command=command.copy(),
            baseline=u0.copy(),
            residual=residual.copy(),
            activity=self.activity,
            gate=self.r,
        )
```

5.4 Proposition 2 (bounded authority), full proof

$\boldsymbol{u}_0 \in \mathcal{U}$. The outer saturation gives $\boldsymbol{u} \in \mathcal{U}$. Componentwise, sat_{ℓ_i} is 1-Lipschitz and fixes $u_{0,i}$, so $|\delta_i| = |\,\mathrm{sat}_{\ell_i}(u_{0,i} + \beta\ell_i s_i) - \mathrm{sat}_{\ell_i}(u_{0,i})| \le |\beta\ell_i s_i| \le \beta\ell_i$ since $|s_i| \le 1$. $\beta = 0$ gives $\boldsymbol{u} = \mathrm{sat}(\boldsymbol{u}_0) = \boldsymbol{u}_0$. □

5.5 Proposition 3 (restricted contraction), full proof, and the clip lemma

Fix $\boldsymbol{a} \neq 0$ and $\boldsymbol{y}^*$, $\lambda_W = 0$, clip and projection inactive, and let $\boldsymbol{z}_k = W_k\boldsymbol{a}$. Then $\boldsymbol{z}_{k+1} = (W_k + \eta\boldsymbol{\varepsilon}_k\boldsymbol{a}^\mathsf{T})\boldsymbol{a} = \boldsymbol{z}_k + \eta\|\boldsymbol{a}\|^2\boldsymbol{\varepsilon}_k$, and $\boldsymbol{\varepsilon}_{k+1} = \boldsymbol{y}^* - \boldsymbol{z}_{k+1} = (1 - \eta\|\boldsymbol{a}\|^2)\boldsymbol{\varepsilon}_k$ P-(27). □

Lemma 1 (Clip never enlarges the PES error)**.** *If $y \in [-1, 1]$ then $|\,\mathrm{clip}_{[-1,1]}(z) - y| \le |z - y|$ for all z; hence $\|\,\mathrm{clip}(\boldsymbol{z}) - \boldsymbol{y}^*\| \le \|\boldsymbol{z} - \boldsymbol{y}^*\|$ for $\boldsymbol{y}^* \in [-1, 1]^m$.*

Proof. If $|z| \le 1$ equality holds. If $z > 1$, $\mathrm{clip}(z) = 1$ and $|1 - y| = 1 - y < z - y$. If $z < -1$ symmetric. The vector case follows coordinatewise. Equivalently, projection onto a closed convex set is nonexpansive and $\boldsymbol{y}^*$ is a fixed point [15]. □

Note: Dr. Sandoval's v7 wording "$\boldsymbol{y}^*$ strictly within $[-1, 1]^m$" is not needed (the closed box suffices, and $\boldsymbol{y}^*$ itself is clipped, so it can equal ± 1). The bound on W comes from $\Pi_{\mathcal{W}}$, not from the readout clip.

6 Latched regime: lagged proportional action and σ-modification

6.1 Constant-basis recursion P-(36)

With $\boldsymbol{a}_k = \bar{\boldsymbol{a}}$ constant and projection inactive, set $\boldsymbol{z}_k = W_k\bar{\boldsymbol{a}}$. From $W_{k+1} = W_k + \eta\boldsymbol{\varepsilon}_k\bar{\boldsymbol{a}}^\mathsf{T} - \lambda_W W_k$,

$$\boldsymbol{z}_{k+1} = (1 - \lambda_W)\boldsymbol{z}_k + \eta\|\bar{\boldsymbol{a}}\|^2[\boldsymbol{y}_k^* - \mathrm{clip}(\boldsymbol{z}_k)].$$

Derived here. When all six gates latch, $\bar{\boldsymbol{a}} = \mathbf{1}$ and $\|\bar{\boldsymbol{a}}\|^2 = 2m = 6$.

6.2 Pole, time constant, DC gain, frequency response

With the clip inactive the recursion is linear: $\boldsymbol{z}_{k+1} = p\,\boldsymbol{z}_k + (1 - p - \lambda_W)\boldsymbol{y}_k^*$, $p = 1 - \lambda_W - \eta\|\bar{\boldsymbol{a}}\|^2$. Its transfer function is $\frac{\eta\|\bar{\boldsymbol{a}}\|^2}{\mathsf{z} - p}$ with

$$p = 0.9952, \quad \tau_f = \frac{h}{1 - p} = 0.104 \text{ s}, \quad \omega_c = \frac{1}{\tau_f} = 9.60 \text{ rad/s}, \quad \text{DC gain} = \frac{\eta\|\bar{\boldsymbol{a}}\|^2}{\eta\|\bar{\boldsymbol{a}}\|^2 + \lambda_W} = 0.99979.$$

For $\omega h \ll 1$ the response is that of $1/(1 + \tau_f s)$: gain $1/\sqrt{1 + (\omega\tau_f)^2}$, lag $\arctan(\omega\tau_f)$.

Why this is not integral action (correcting the v7 highlight). An integrator has a pole at $z = 1$ and infinite DC gain; here $p < 1$ and the DC gain is ≈ 1, so the residual converges to $\beta\ell_i\, y_i^* = -\beta\ell_i k_e\, e_{q,i}$: proportional, low-pass filtered. It does not remove steady-state error by itself.

Added stiffness. The PD stiffness is $k_{q,i} = 2j_i\omega_{n,i}^2$; the latched residual adds $\beta k_e\ell_i = \beta k_e j_i \nu_i$ (while $|k_e e_{q,i}| < 1$), a ratio

$$\frac{\beta k_e \ell_i}{k_{q,i}} = \frac{\beta k_e \nu_i}{2\omega_{n,i}^2} = \frac{0.5 \cdot 8 \cdot 12}{2 \cdot 16} = 1.50 \text{ (roll, pitch)}, \qquad \frac{0.5 \cdot 8 \cdot 10}{2 \cdot 12.25} = 1.63 \text{ (yaw)},$$

independent of the vehicle. *Derived here; verified in the script.*

Reference content. Roll, Full3 and Hard45 are sums of constant-amplitude, constant-frequency AM–FM components, whose instantaneous frequencies are the component frequencies [16]: at most 0.7, 1.0 and 2.10 rad/s. At 2.10 rad/s the filter gain is 0.977 and the lag 12.3° (Roll: 0.997, 4.2°; Full3: 0.995, 5.9°). Yaw360 is a smoothstep whose yaw rate peaks at $2\pi \cdot 1.5/45 = 0.209$ rad/s.

Graded-tail regime. With every unit at rest, $\|\bar{\boldsymbol{a}}\|^2 = 6 \cdot 0.3727^2 = 0.833$, giving $p = 0.99933$ and $\tau_f = 0.75$ s. (Activities are not exactly constant there, so this is an approximation.)

`experiments/theory_certificate_2026-09-12/compute_local_certificate.py`: 149–162 regimes, lag, stiffness ratio

```
    # latched-PES filter and graded-tail regime [eq:constantbasis]
    a_rest = min(1.0, G_READOUT / (1 + np.exp(-ALPHA_H * (I_B - V_TH))))
    regimes = {}
    for name, a2 in (("latched", 6.0), ("tail", 6 * a_rest**2)):
        pole = 1 - LAMBDA_W - ETA * a2
        tau_f = H_STEP / (1 - pole)
        regimes[name] = dict(a_norm2=a2, pole=pole, tau_s=tau_f, corner_rad_s=1 / tau_f,
                             dc_gain=ETA * a2 / (ETA * a2 + LAMBDA_W),
                             leak_mismatch=LAMBDA_W / (ETA * a2))
    lat = regimes["latched"]
    lag = {m: dict(gain=1 / np.sqrt(1 + (w * lat["tau_s"])**2),
                   phase_deg=float(np.degrees(np.arctan(w * lat["tau_s"]))))
           for m, w in MISSION_MAX_FREQ.items()}
    stiffness_ratio = BETA * K_E * ell / kq                    # = beta k_e nu / (2 wn^2)
```

6.3 σ-modification fixed point

At a fixed point of P-(25) with constant $\bar{\boldsymbol{a}}$ and inactive clip/projection, $0 = \eta\boldsymbol{\varepsilon}\bar{\boldsymbol{a}}^\mathsf{T} - \lambda_W W$. Right-multiplying by $\bar{\boldsymbol{a}}$: $\eta\|\bar{\boldsymbol{a}}\|^2\boldsymbol{\varepsilon} = \lambda_W \boldsymbol{s}$, so $\|\boldsymbol{\varepsilon}\| = \frac{\lambda_W}{\eta\|\bar{\boldsymbol{a}}\|^2}\|\boldsymbol{s}\|$: $2.1 \times 10^{-4}\,\|\boldsymbol{s}\|$ latched and $1.5 \times 10^{-3}\,\|\boldsymbol{s}\|$ at the tail. This is an *equality* for the PES mismatch $\boldsymbol{y}^* - \boldsymbol{s}$, not a bound on the tracking error θ_q (correcting the v7 wording).

6.4 Why silent bases do not learn (Sec. IV-D, neuron families)

Proposition 3's factor $1 - \eta\|\boldsymbol{a}\|^2 \approx 1$ when $\boldsymbol{a} \approx 0$. The benchmark's occupancy metric is the fraction of *unit-samples* with $|a_j| > 10^{-9}$:

`experiments/nengo_nef_replication_2026-08-29/compile_neuron_family_benchmark.py`: 86–93 occupancy, drift, update RMS

```
                record.update({
                    "activity_mean": float(np.mean(activity)),
                    "activity_max": float(np.max(activity)),
                    "activity_occupancy_fraction": float(np.mean(np.abs(activity) > 1e-9)),
                    "decoder_drift_fro": float(np.linalg.norm(weights[-1] - weights[0])),
                    "decoder_final_fro": float(np.linalg.norm(weights[-1])),
                    "weight_update_rms": float(np.sqrt(np.mean(updates ** 2))),
                })
```

Family (both vehicles)	Full3	Hard45	Hardstep	Roll	Yaw360
LIF / ALIF / LIFRate occupancy	0.001	0.001	0.06–0.20	< 0.0005	< 0.0005
LIF / ALIF / LIFRate PES gain [%]	≤ 0.06	≤ 0.09	7–18	0	0
CMG occupancy / PES gain [%]	0.999 / 56	0.999 / 53	0.999 / 18–21	0.999 / 17	0.999 / 61
Izhikevich occupancy / PES gain [%]	1.0 / 52	1.0 / 50	1.0 / 21–23	1.0 / 51	1.0 / 56–61

Persistent activity is necessary for learning, but Izhikevich–PES has it too while spending more effort, so contraction alone does not explain CMG's effort advantage (correcting the v7 claim that it is "the formal reason CMG breaks the tradeoff").

7 Allocation-aware local stability (Sec. III-D)

7.1 Secant lemma and L_A P-(29)

For scalar $\mathrm{clip}_{[0,f_{\max}]}$, which is monotone and 1-Lipschitz, $\mathrm{clip}(x) - \mathrm{clip}(y) = d\,(x-y)$ with $d \in [0,1]$ (take $d = 0$ if $x = y$). At the same state and collective, $\boldsymbol{f}_{\mathrm{act}}(\boldsymbol{u}) - \boldsymbol{f}_{\mathrm{act}}(\boldsymbol{u}_0) = D\,B^{-1}E\boldsymbol{\delta}$, and the torque difference is $HBDB^{-1}E\boldsymbol{\delta}$. $D \mapsto \|HBDB^{-1}E\|_2$ is convex (a norm of an affine map), so its maximum over the box $[0,1]^4$ is attained at one of the 16 vertices. *Derived here.*

Numbers. $L_A = 4.53$ (X2), 3.33 (CF2). At hover collective, every vertex of the torque box $\mathcal{U}$ is rotor-feasible for CF2 (so $D = I$ and 1 replaces L_A locally) but not for X2.

`experiments/theory_certificate_2026-09-12/compute_local_certificate.py`: 122–130 L_A over 16 vertices; hover feasibility of $\mathcal{U}$

```
    E = np.vstack([np.zeros((1, 3)), np.eye(3)])
    H = E.T
    LA = max(np.linalg.norm(H @ B @ np.diag(d) @ Binv @ E, 2)
             for d in itertools.product((0.0, 1.0), repeat=4))

    # rotor feasibility of the whole torque box at hover collective
    f_hover = [Binv @ np.r_[mass * g, np.array(s) * ell] for s in itertools.product((-1, 1), repeat=3)]
    f_hover = np.array(f_hover)
    box_feasible = bool(np.all(f_hover >= 0) and np.all(f_hover <= fmax))
```

7.2 Local model P-(30)–P-(31)

Constant reference ($\boldsymbol{\omega}_d = 0$), smooth branch $q_{e,0} = \sqrt{1 - \|\boldsymbol{e}_q\|^2}$. From (3), $\dot{\boldsymbol{e}}_q = \frac{1}{2}\boldsymbol{\omega} + \frac{1}{2}[(\sqrt{1-\|\boldsymbol{e}_q\|^2} - 1)\boldsymbol{\omega} + \boldsymbol{e}_q \times \boldsymbol{\omega}]$. From P-(2), add and subtract the ideal PD torque: $\boldsymbol{J}\dot{\boldsymbol{\omega}} = -K_q\boldsymbol{e}_q - K_\omega\boldsymbol{\omega} + \boldsymbol{d} - \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}$ with $\boldsymbol{d} = \boldsymbol{\tau}_{\mathrm{act}} + \boldsymbol{\tau}_{\mathrm{p}} + \boldsymbol{\tau}_{\mathrm{d}} + K_q\boldsymbol{e}_q + K_\omega\boldsymbol{\omega}$. Multiplying by $\boldsymbol{J}^{-1}$ gives A, F and $\boldsymbol{g}$ as in the paper. *Derived here.*

7.3 Quadratic-growth constant L_r P-(32)

First block. $\boldsymbol{\alpha} = (\sqrt{1-s} - 1)\boldsymbol{\omega}$ with $s = \|\boldsymbol{e}_q\|^2$ is parallel to $\boldsymbol{\omega}$; $\boldsymbol{e}_q \times \boldsymbol{\omega} \perp \boldsymbol{\omega}$. So $\|\boldsymbol{\alpha} + \boldsymbol{e}_q \times \boldsymbol{\omega}\|^2 = (1 - \sqrt{1-s})^2\|\boldsymbol{\omega}\|^2 + \|\boldsymbol{e}_q \times \boldsymbol{\omega}\|^2$. For $s \in [0,1]$, $\sqrt{1-s} \geq 1 - s$, so $0 \leq 1 - \sqrt{1-s} \leq s \leq \sqrt{s}$, i.e. $(1 - \sqrt{1-s})^2 \leq s$. With $\|\boldsymbol{e}_q \times \boldsymbol{\omega}\| \leq \|\boldsymbol{e}_q\|\|\boldsymbol{\omega}\|$ the first block is at most $\frac{1}{2}\sqrt{2}\,\|\boldsymbol{e}_q\|\|\boldsymbol{\omega}\| \leq \frac{\sqrt{2}}{4}\|\boldsymbol{x}\|^2$ (AM–GM: $\|\boldsymbol{e}_q\|\|\boldsymbol{\omega}\| \leq \frac{1}{2}\|\boldsymbol{x}\|^2$).

Second block (Dr. Sandoval's bound). $\|\boldsymbol{J}^{-1}(\boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega})\| \leq \|\boldsymbol{J}^{-1}\|\,\|\boldsymbol{\omega}\|\,\|\boldsymbol{J}\|\,\|\boldsymbol{\omega}\| = \kappa(\boldsymbol{J})\|\boldsymbol{\omega}\|^2$.

Second block (refined here). Since $\boldsymbol{\omega} \times \boldsymbol{\omega} = 0$, $\boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega} = \boldsymbol{\omega} \times (\boldsymbol{J} - \mu I)\boldsymbol{\omega}$ for every $\mu \in \mathbb{R}$. With $\mu = \frac{1}{2}(\lambda_{\max} + \lambda_{\min})$, $\|\boldsymbol{J} - \mu I\| = \frac{1}{2}(\lambda_{\max} - \lambda_{\min})$ and $\|\boldsymbol{J}^{-1}\| = 1/\lambda_{\min}$, so the block is at most $\frac{1}{2}(\kappa(\boldsymbol{J}) - 1)\|\boldsymbol{\omega}\|^2$, which vanishes for isotropic inertia (no gyroscopic torque), as it should.

Combine. $\|\boldsymbol{g}\| \leq \|\mathrm{block}_1\| + \|\mathrm{block}_2\| \leq L_r\|\boldsymbol{x}\|^2$ with $L_r = \frac{\sqrt{2}}{4} + \frac{1}{2}(\kappa - 1)$ (or $\frac{\sqrt{2}}{4} + \kappa$).

	$\kappa(\boldsymbol{J})$	L_r (v7)	L_r (refined)	sampled sup $\|\boldsymbol{g}\|/\|\boldsymbol{x}\|^2$	spectral abscissa of A
X2	2.389	2.743	1.048	0.483	-1.395
CF2	1.351	1.704	0.529	0.283	-3.325

The sampled supremum uses 2×10^5 random points in the unit ball; both bounds hold.

`experiments/theory_certificate_2026-09-12/compute_local_certificate.py`: 110–120 κ, both L_r, sampled supremum

```
    evJ = np.linalg.eigvalsh(J)
    kappa = evJ[-1] / evJ[0]
    Lr_steven = np.sqrt(2) / 4 + kappa                          # highlighted v7 bound
    Lr_tight = np.sqrt(2) / 4 + (kappa - 1) / 2                 # centred-inertia refinement

    # empirical sup of ||g||/||x||^2 on the unit ball (checks both bounds)
    worst = 0.0
    for _ in range(200_000):
        x = rng.normal(size=6)
        x *= rng.uniform(1e-3, 0.999) / np.linalg.norm(x)
        worst = max(worst, np.linalg.norm(remainder(x, J, Jinv)) / (x @ x))
```

7.4 A is Hurwitz (energy + LaSalle)

Take $V_L = \boldsymbol{e}_q^{\mathsf{T}} K_q \boldsymbol{e}_q + \frac{1}{2}\boldsymbol{\omega}^{\mathsf{T}} \boldsymbol{J} \boldsymbol{\omega}$ (positive definite). Along $\dot{\boldsymbol{e}}_q = \frac{1}{2}\boldsymbol{\omega}$, $\boldsymbol{J}\dot{\boldsymbol{\omega}} = -K_q \boldsymbol{e}_q - K_\omega \boldsymbol{\omega}$: $\dot{V}_L = 2\boldsymbol{e}_q^{\mathsf{T}} K_q \cdot \frac{1}{2}\boldsymbol{\omega} + \boldsymbol{\omega}^{\mathsf{T}}(-K_q \boldsymbol{e}_q - K_\omega \boldsymbol{\omega}) = -\boldsymbol{\omega}^{\mathsf{T}} K_\omega \boldsymbol{\omega} \le 0$, using $K_q = K_q^{\mathsf{T}}$ (and $\boldsymbol{J} = \boldsymbol{J}^{\mathsf{T}}$; off-diagonal products of inertia are allowed). On $\{\dot{V}_L = 0\}$, $\boldsymbol{\omega} \equiv 0 \Rightarrow \dot{\boldsymbol{\omega}} \equiv 0 \Rightarrow K_q \boldsymbol{e}_q = 0 \Rightarrow \boldsymbol{e}_q = 0$. By LaSalle's invariance principle [17] the origin of the linear system is globally asymptotically stable, so A is Hurwitz, and for every $Q_L \succ 0$ the Lyapunov equation has a unique $P \succ 0$ [17]. (v7 said "linear momentum"; it is the angular-momentum equation.)

`experiments/theory_certificate_2026-09-12/compute_local_certificate.py`: 59–65 Lyapunov equation by vectorization

```
def lyap(A, Q):
    """Solve A^T P + P A = -Q by vectorization (6x6, exact to rounding)."""
    n = A.shape[0]
    I = np.eye(n)
    M = np.kron(I, A.T) + np.kron(A.T, I)
    P = np.linalg.solve(M, -Q.reshape(-1, order="F")).reshape(n, n, order="F")
    return 0.5 * (P + P.T)
```

7.5 Theorem 1, full proof

On $\|\boldsymbol{x}\| \le r < 1$:

$$\begin{aligned}\dot{V} &= \boldsymbol{x}^{\mathsf{T}}(A^{\mathsf{T}}P + PA)\boldsymbol{x} + 2\boldsymbol{x}^{\mathsf{T}}P\boldsymbol{g} + 2\boldsymbol{x}^{\mathsf{T}}PF\boldsymbol{d} \le -\lambda_{\min}(Q_L)\|\boldsymbol{x}\|^2 + 2\|P\|L_r\|\boldsymbol{x}\|^3 + 2b\|\boldsymbol{x}\|\bar{d} \\ &\le -c\|\boldsymbol{x}\|^2 + 2b\|\boldsymbol{x}\|\bar{d} \le -\tfrac{c}{2}\|\boldsymbol{x}\|^2 + \tfrac{2b^2}{c}\bar{d}^2 \qquad \text{(Young: } 2b\|\boldsymbol{x}\|\bar{d} \le \tfrac{c}{2}\|\boldsymbol{x}\|^2 + \tfrac{2b^2}{c}\bar{d}^2\text{).}\end{aligned}$$

Invariance. $p_-\|\boldsymbol{x}\|^2 \le V \le p_+\|\boldsymbol{x}\|^2$, so $\{V \le p_- r^2\} \subset \{\|\boldsymbol{x}\| \le r\}$. On the boundary $V = p_- r^2$, $\|\boldsymbol{x}\|^2 \ge p_- r^2/p_+$, and $\dot{V} < 0$ iff $\bar{d}^2 < \frac{c^2}{4b^2}\|\boldsymbol{x}\|^2$, which holds under P-(33). *Bound.* $\dot{V} \le -\frac{c}{2p_+}V + \frac{2b^2}{c}\bar{d}^2$; the comparison lemma [17] gives $V(t) \le e^{-\alpha t}V(0) + \frac{2b^2\bar{d}^2/c}{\alpha} = e^{-\alpha t}V(0) + \frac{4p_+b^2}{c^2}\bar{d}^2$, $\alpha = c/(2p_+)$. This is local input-to-state stability [18]. □

7.6 Optimal radius, maximal admissible disturbance, and $\beta_{\max}$ P-(35)

The admissible bound is $\bar{d} < \phi(r)\frac{1}{2b}\sqrt{p_-/p_+}$ with $\phi(r) = (\lambda - 2\|P\|L_r r)r$, $\lambda = \lambda_{\min}(Q_L)$. $\phi'(r) = \lambda - 4\|P\|L_r r = 0$ at $r^* = \lambda/(4\|P\|L_r)$, where $c = \lambda/2 > 0$ and $\phi(r^*) = \lambda^2/(8\|P\|L_r)$. If $r^* \ge 1$ the

constraint $r < 1$ binds instead. Hence $\bar{d}_{\max} = \frac{\lambda^2}{16\, b \|P\| L_r} \sqrt{p_-/p_+}$. With $\bar{d} = \bar{d}_0 + L_A \beta \|\boldsymbol{\ell}\|$ (Sec. 7.1), $\beta < (\bar{d}_{\max} - \bar{d}_0)/(L_A \|\boldsymbol{\ell}\|)$, meaningful only if $\bar{d}_0 < \bar{d}_{\max}$. $\bar{d}_{\max}$ is invariant to scaling Q_L (P, b, λ scale together); only its shape matters.

Justifying $\bar{d}_0 = 0$ at hover. Nominal model ($\boldsymbol{\tau}_{\mathrm{p}} = \boldsymbol{\tau}_{\mathrm{d}} = 0$); inside the certified ball $r^* \sqrt{k_{q,i}^2 + k_{\omega,i}^2} \leq \ell_i$ (PD unsaturated, checked in the script), and the resulting rotor forces stay inside $[0, \boldsymbol{f}_{\max}]$ at hover collective. Sampled-data and solver errors are excluded, as the paper states.

	r^*	$\bar{d}_{\max}$ [N m]	$\beta_{\max}$ (with L_A)	$\beta_{\max}$ (exact allocation)	experiment
X2	5.31×10^{-2}	3.90×10^{-4}	9.69×10^{-5}	4.39×10^{-4}	$\beta = 0.5$
CF2	1.05×10^{-1}	1.11×10^{-6}	6.43×10^{-4}	2.14×10^{-3}	$\beta = 0.5$

With the v7 constant $L_r = \sqrt{2}/4 + \kappa$ the margins are 2.6× (X2) and 3.2× (CF2) smaller.

`experiments/theory_certificate_2026-09-12/compute_local_certificate.py`: 78–89 certificate: r^*, c, b, $\bar{d}_{\max}$

```python
def certificate(A, F, Lr, Q):
    P = lyap(A, Q)
    ev = np.linalg.eigvalsh(P)
    p_min, p_max = ev[0], ev[-1]
    Pn = np.linalg.norm(P, 2)
    b = np.linalg.norm(P @ F, 2)
    lq = np.linalg.eigvalsh(Q)[0]
    r_star = min(lq / (4 * Pn * Lr), 1.0 - 1e-9)
    c = lq - 2 * Pn * Lr * r_star
    dbar = c * r_star / (2 * b) * np.sqrt(p_min / p_max)
    return dict(P=P, p_min=p_min, p_max=p_max, P_norm=Pn, b=b, lam_Q=lq,
                r_star=r_star, c=c, dbar_max=dbar, alpha=c / (2 * p_max))
```

`experiments/theory_certificate_2026-09-12/compute_local_certificate.py`: 132–147 Q_L shape scan, PD saturation check, $\beta_{\max}$

```python
    base = certificate(A, F, Lr_tight, np.eye(6))
    steven = certificate(A, F, Lr_steven, np.eye(6))
    # Q_L shape search (the margin is invariant to scaling Q_L)
    best = base | {"q_e": 1.0}
    for qe in np.logspace(-3, 3, 241):
        cand = certificate(A, F, Lr_tight, np.diag([qe] * 3 + [1.0] * 3))
        if cand["dbar_max"] > best["dbar_max"]:
            best = cand | {"q_e": float(qe)}

    # PD saturation inactive inside the certified ball?
    pd_gain_row = np.sqrt(kq**2 + kw**2)
    pd_unsat = bool(np.all(best["r_star"] * pd_gain_row <= ell))

    ell_norm = np.linalg.norm(ell)
    beta_max = best["dbar_max"] / (LA * ell_norm)
    beta_max_exact_alloc = best["dbar_max"] / ell_norm
```

7.7 Remark (sign selection on certified trajectories)

$\|\boldsymbol{e}_q\| \leq \|\boldsymbol{x}\| \leq r < 1 \Rightarrow q_{e,0}^2 = 1 - \|\boldsymbol{e}_q\|^2 \geq 1 - r^2 > 0$. The branch never reaches $q_{e,0} = 0$, so the smooth-branch model used in the proof is self-consistent. A measurement perturbation of $q_{e,0}$ smaller than $\sqrt{1 - r^2}$ cannot flip sgn_+. This is local: it does not contradict [3], and it does not make the controller globally robust near $q_{e,0} = 0$ [5] (the v7 phrase "robustly avoiding unwinding" overstates it).

8 Protocol, metrics, noise, and disturbance

8.1 Metric

$\bar{\theta}_q = \frac{1}{N}\sum_k \theta_{q,k}$ over all $N = 90{,}000$ control steps (45 s at $h = 5 \times 10^{-4}$); reduction $= 100(\bar{\theta}_{q,\mathrm{PD}} - \bar{\theta}_{q,\mathrm{CMG}})/\bar{\theta}_{q,\mathrm{PD}}$.

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py`: 346–347 per-step error

```
346             qerr_value = quat_angle(qe)
347             metric_qerr.append(qerr_value)
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py`: 549–551 summary statistics

```
549         "qerr_mean_rad": float(np.mean(qerr)),
550         "qerr_max_rad": float(np.max(qerr)),
551         "qerr_rmse_rad": float(np.sqrt(np.mean(qerr**2))),
```

8.2 Campaign settings

`experiments/nengo_nef_replication_2026-08-29/run_neuron_family_campaign.py`: 50–58 matched runner arguments (β, η, k_e, k_ω^*, h, scale)

```
50                 command = [
51                     sys.executable, str(HERE / "run_mujoco_nef.py"),
52                     "--platform", platform, "--controller", controller,
53                     "--mission", mission, "--duration", str(args.duration),
54                     "--dt", "0.0005", "--beta", "0.5",
55                     "--learning-rate", "0.0008", "--k-error", "8", "--k-rate", "0",
56                     "--actuator-scale", str(PLATFORM_SCALE[platform]),
57                     "--log-every", str(args.log_every), "--output", str(output),
58                 ]
```

8.3 Measurement noise (Sec. IV-C)

A small-angle quaternion from i.i.d. Gaussian Euler angles is left-multiplied into q, and Gaussian noise is added to $\boldsymbol{\omega}$ (listing in Sec. 1, lines 220–224). θ_q is computed from the *corrupted* q, so the noisy columns measure noisy-feedback error; the floor for isotropic attitude noise σ is roughly $\sigma\sqrt{8/\pi}$.

8.4 Disturbance (Sec. IV-F)

$\boldsymbol{\tau}_\mathrm{d}(t) = d_b\rho(t)[\sin(1.31t + .2),\ .8\sin(1.73t + 1.1),\ .6\sin(2.17t + 2)]^\mathsf{T}$ in body axes, applied as a world-frame torque $R\boldsymbol{\tau}_\mathrm{d}$ through `xfrc_applied`.

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py`: 299–307 body-frame multi-sine disturbance

```
299             if args.disturbance_torque > 0:
300                 disturbance_body = args.disturbance_torque * np.array(
301                     [
302                         np.sin(1.31 * t + 0.2),
303                         0.8 * np.sin(1.73 * t + 1.1),
304                         0.6 * np.sin(2.17 * t + 2.0),
305                     ]
306                 )
307                 data.xfrc_applied[body_id, 3:6] = R @ disturbance_body
```

$\rho(t)$: zero during warm-up, then on/off with raised-cosine edges $\rho = \frac{1}{2}(1 - \cos(\pi\,\xi/t_r))$ over ramp $t_r = 0.25$ s:

experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_perturbed.py : 88–117 intermittent envelope $\rho(t)$

```python
def disturbance_gain(t: float, args) -> float:
    """Scale on the disturbance torque at time ``t``.

    ``continuous``   always 1.0 (canonical behaviour).
    ``intermittent`` 0.0 for the first ``--disturbance-warmup`` seconds, then
                     alternating ON for ``--disturbance-on`` and OFF for
                     ``--disturbance-off``. Each edge is smoothed over
                     ``--disturbance-ramp`` seconds with a raised cosine so
                     the plant sees a finite slew rather than an impulse.
    """
    if args.disturbance_schedule == "continuous":
        return 1.0

    warm = args.disturbance_warmup
    if t < warm:
        return 0.0

    period = args.disturbance_on + args.disturbance_off
    phase = (t - warm) % period
    ramp = max(args.disturbance_ramp, 1e-9)

    if phase < args.disturbance_on:
        # rising edge at phase 0, falling edge at phase = on
        if phase < ramp:
            return 0.5 * (1.0 - np.cos(np.pi * phase / ramp))
        if phase > args.disturbance_on - ramp:
            x = (args.disturbance_on - phase) / ramp
            return 0.5 * (1.0 - np.cos(np.pi * x))
        return 1.0
    return 0.0
```

9 What changed relative to the highlighted v7, mathematically

1. **Gate step.** Exponential Euler is correct and unconditionally convex, but it is not the implemented scheme; kept as a cited remark, the paper keeps the implemented forward Euler with clip, and Proposition 1 states both regimes of h.

2. **Allocation.** $B^\dagger = B^{-1}$ for four rotors; code uses `np.linalg.inv`. L_A now has numbers, and the CF2 hover-feasibility case replaces L_A by 1. The weighted-norm idea needs the theorem restated in weighted coordinates; the symbol W (decoder) is avoided.

3. **Remainder constant.** v7's $\sqrt{2}/4 + \kappa$ is valid; the centred-inertia bound $\sqrt{2}/4 + \frac{1}{2}(\kappa - 1)$ is tighter. It is a quadratic-growth constant, not a Lipschitz constant, and not "exact".

4. **Hurwitz proof.** v7's energy/LaSalle argument is correct; wording fixed (symmetric $\boldsymbol{J}$ with products of inertia; angular momentum).

5. **Optimal radius, $\beta_{\max}$.** Algebra confirmed; $r^* < 1$ caveat added; renamed from "Ultimate Robustness Margin" ("ultimate bound" already means something else in ISS); numbers show the certificate is about three orders of magnitude below $\beta = 0.5$, so "maximum mathematically safe authority" must read "sufficient".

6. **Topological remark.** Correct as a local statement; "robustly avoiding unwinding" removed.

7. **Latched PES.** First-order low-pass of the proportional target, pole 0.9952, DC gain ≈ 1: lagged proportional action adding 1.5–1.63× PD stiffness, not integral action.

8. **σ-modification.** Analogy correct; the relation is an equality for the PES mismatch, not a bound on tracking error; "permanently resolving" removed.

9. **Silent bases.** Confirmed with the occupancy data; the claim that it is "the formal reason CMG breaks the tradeoff" is not supported (Izhikevich has persistent activity too).

10. **Projection lemma.** Correct; "strictly within" unnecessary; weight boundedness is from $\Pi_{\mathcal{W}}$.

A Source files and hashes

File	SHA-256 (16)	Lines
`src/gra_control/math3d.py`	`31620612f184a22f`	83
`src/gra_control/gate.py`	`ea6d7853fff724d2`	143
`src/gra_control/platforms/drone.py`	`50af6d1f6de51788`	271
`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py`	`fbfd069d78d4040e`	362
`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py`	`10e50c274010aa97`	629
`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_perturbed.py`	`896b70adceb124dc`	686
`experiments/nengo_nef_replication_2026-08-29/compile_neuron_family_benchmark.py`	`189c39d8c467c278`	201
`experiments/nengo_nef_replication_2026-08-29/run_neuron_family_campaign.py`	`e39400eeb9068950`	90
`experiments/theory_certificate_2026-09-12/compute_local_certificate.py`	`b7eb811f74600c6d`	260

References

[1] J. Diebel, "Representing attitude: Euler angles, unit quaternions, and rotation vectors," *Matrix*, vol. 58, no. 15–16, pp. 1–35, 2006.

[2] H. Schaub and J. L. Junkins, *Analytical Mechanics of Space Systems*, 4th ed. AIAA, 2018.

[3] S. P. Bhat and D. S. Bernstein, "A topological obstruction to continuous global stabilization of rotational motion and the unwinding phenomenon," *Systems & Control Letters*, vol. 39, no. 1, pp. 63–70, 2000.

[4] N. A. Chaturvedi, A. K. Sanyal, and N. H. McClamroch, "Rigid-body attitude control," *IEEE Control Systems Magazine*, vol. 31, no. 3, pp. 30–51, 2011.

[5] C. G. Mayhew, R. G. Sanfelice, and A. R. Teel, "Quaternion-based hybrid control for robust global attitude tracking," *IEEE Transactions on Automatic Control*, vol. 56, no. 11, pp. 2555–2566, 2011.

[6] I. Rubio Scola, O. A. García Alcántara, S. Sandoval, E. S. Espinoza Quesada, H. Haimovich, and L. R. Garcia Carrillo, "Globally asymptotically stable trajectory tracking of underactuated UAVs using geometric algebra," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 62, pp. 1491–1508, 2026.

[7] L. Escamilla, L. R. Garcia Carrillo, S. Sandoval, and E. S. Espinoza Quesada, "Stabilization and trajectory tracking of a subactuated aircraft based on a geometric algebra approach," in *2023 American Control Conference (ACC)*, 2023, pp. 3233–3238.

[8] L. R. García Carrillo, A. E. Dzul López, R. Lozano, and C. Pégard, *Quad Rotorcraft Control: Vision-Based Hovering and Navigation*, ser. Advances in Industrial Control. London: Springer, 2013.

[9] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *The Journal of Physiology*, vol. 117, no. 4, pp. 500–544, 1952.

[10] M. Hochbruck and A. Ostermann, "Exponential integrators," *Acta Numerica*, vol. 19, pp. 209–286, 2010.

[11] D. MacNeil and C. Eliasmith, "Fine-tuning and the stability of recurrent neural networks," *PLoS ONE*, vol. 6, no. 9, p. e22885, 2011.

[12] A. R. Voelker, "A solution to the dynamics of the prescribed error sensitivity learning rule," Centre for Theoretical Neuroscience, University of Waterloo, Tech. Rep. CTN-TR-20151001-01, Oct. 2015. [Online]. Available: https://www.compneuro.uwaterloo.ca/files/publications/voelker.2015.pdf

[13] P. A. Ioannou and P. V. Kokotović, "Instability analysis and improvement of robustness of adaptive control," *Automatica*, vol. 20, no. 5, pp. 583–594, 1984.

[14] P. A. Ioannou and J. Sun, *Robust Adaptive Control*. Upper Saddle River, NJ: Prentice Hall, 1996.

[15] H. H. Bauschke and P. L. Combettes, *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*, 2nd ed. Cham: Springer, 2017.

[16] S. Sandoval and P. L. De Leon, "The instantaneous spectrum: A general framework for time-frequency analysis," *IEEE Transactions on Signal Processing*, vol. 66, no. 21, pp. 5679–5693, 2018.

[17] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2002.

[18] E. D. Sontag, "Smooth stabilization implies coprime factorization," *IEEE Transactions on Automatic Control*, vol. 34, no. 4, pp. 435–443, 1989.